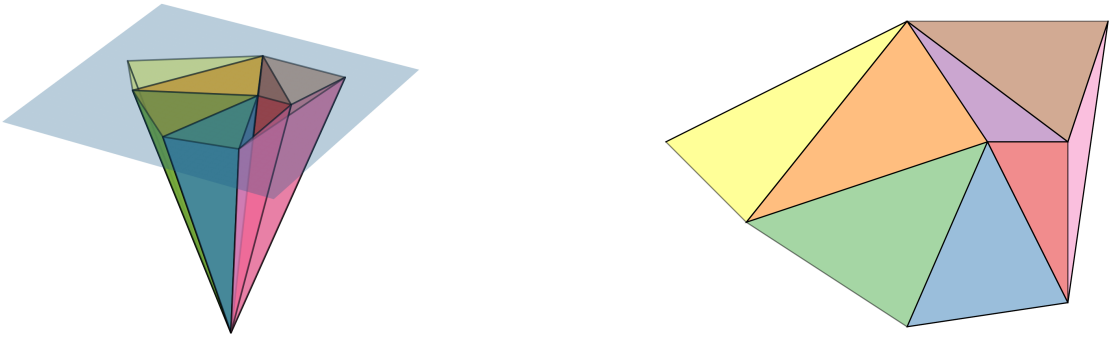
Solving Two-Stage Black-Box Linear Optimization with a Zero-order oracle

Hippolyte Wallaert

July 31, 2025

Abstract

We study an instance of two-stage linear optimization problems in which the recourse function is a black box. We focus on the case where the recourse function is a linear program with an unknown cost vector, accessible only through a zero-order oracle. We propose an exact solution method that constructs Benders-type cuts by systematically querying the oracle at carefully selected neighboring points. To improve computational efficiency, we derive algebraic conditions that reduce the number of oracle calls required at each iteration. The proposed approach is evaluated on a facility location instance, demonstrating its effectiveness and computational performance. We conclude by outlining potential extensions and applications of the method.



(a) Critical regions in the RHS space.

(b) Critical regions in the x-space.

Figure 1: Our method exploits regions where the downstream cost is affine, recovering local slopes from carefully chosen oracle evaluations.

1 Introduction

Recent progress in warehouse automation is bringing significant advantages to companies that adopt robotic warehousing, as highlighted by The Boston Consulting Group [5]. The logistics sector is at the forefront of this transformation. The largest facilities now operate at unprecedented scales, deploying fleets of several thousand robots.

The organization of automated warehouses can be summarized as follows: a central storage area contains shelves with items, and automated pods transport these shelves to workstations where human operators assemble orders. The standard decision process involves receiving a set of orders, assigning shelves to workstations (first-stage decision), and subsequently routing the pods within the warehouse to carry out these assignments (second-stage decision). Given the large scale of operations, optimization now plays a crucial role in improving overall warehouse performance. Various synergies can be exploited to enhance efficiency. For instance, different orders may require the same item, and each shelf may contain multiple items. This makes non-trivial, optimization-driven shelf assignments highly beneficial, as they can substantially reduce order preparation times compared to naive strategies such as always selecting the closest shelf for each item. A further challenge arises in routing the pods once shelf assignments have been determined, where effective routing is needed to avoid traffic and congestion. The scale of these problems has grown so dramatically in practice that solving them under strict time constraints has become a major challenge, and research on effective solution methods continues to evolve.



Figure 2: Fulfillment center featuring workstations and inventory pods. [1]

A crucial aspect is that these decisions are sequential and cannot be solved independently. In fact, pod assignments can be improved if the decision process is aware of the congestion constraints that shape the lower-level routing problem. Our work aims to build a framework for bilevel optimization of this kind. Instead of solving the first-stage problem on its own, solving our bilevel reformulation will lead to better practical decisions. While it does not yet provide a full solution to the entire problem, it completely solves a useful sub-formulation that can be extended toward a comprehensive framework.

Our contributions are summarized as follows:

- We introduce our bilevel optimization problem, in which the upper level is a continuous

[1] Illustrations and caption taken from Felipe Cordera's article yet to be published.

linear program and the lower level is also a continuous linear program, but with an unknown cost vector.

- We construct an algorithm solving the problem to optimality, assuming access to a zero-order oracle of the recourse function. The method constructs a lower approximation of the recourse function by generating Benders-type cuts, which are derived using directional derivatives.
- We exploit algebraic dependencies to reuse previous evaluations and significantly reduce oracle calls.
- We provide preliminary computational results of this method on a facility location instance.

Finally, we outline directions for future work needed to build a complete framework, which we hope will contribute to improving current solutions for the pod assignment problem.

2 Related work

Robotic warehousing operations. This study builds upon the introductory contribution of Barnhart, Jacquillat, and Schmid [1], who investigated robotic warehousing operations. Further work, currently unpublished, has been carried out by Felipe Cordera, focusing on the assignment of pods. In his approach, the assignment problem is formulated as a linear program (TAP), and several results were derived to enable its resolution within a strict three-minute computational time limit. Nevertheless, these solutions sometimes proved less effective than expected due to congestion effects arising during pod routing. Introducing congestion-penalty terms into the TAP objective function, derived from heuristic considerations, was shown to mitigate these issues and reduce operational runtimes. This observation provides the main motivation for the present study, which seeks to develop a new framework to improve TAP decisions.

Bilevel optimization. Bilevel optimization provides a natural framework for large-scale, hierarchically structured decision problems. As problem instances grow, it often becomes necessary to decompose the decision process into successive stages, since solving the full problem directly is no longer tractable. The range of applications of bilevel optimization is extremely broad, and consequently the literature on this topic is extensive. In our setting, the lower-level problem defines a piecewise linear convex function. This naturally directs attention to cutting-plane and outer-approximation methods, which have formed the methodological backbone of convex optimization since the seminal work of Kelley [11]. Decomposition approaches have played a central role in bilevel optimization, with Benders decomposition [2] serving as a fundamental tool applied across a wide variety of domains.

Multi-parametric programming. Our approach is conceptually close to the multiparametric programming literature, which aims at solving linear programs where the objective and/or the constraints are subject to vary linearly from a given paramter. Foundational work by Gal and Nedoma [7] established the basis for this theory, while more recent surveys, such as Pistikopoulos et al. [13], provide a comprehensive overview of state-of-the-art strategies. These insights have

been very useful to develop our method, even though our final solution does not borrow any of the algorithms developed by this literature.

Two-stage stochastic optimization. It is also important to distinguish our setting from the well-developed literature on two-stage stochastic optimization. In stochastic programming, the uncertainty in the cost vector or right-hand side is modeled through probability distributions, and the objective is typically an expectation over scenarios. Solution methods such as Sample Average Approximation (SAA) , stochastic Benders decomposition, and robust optimization [14] rely on the existence of either a confidence set for the uncertain parameters or repeated sampling to approximate the expected recourse function. In contrast, our framework is deterministic: the lower-level problem is fixed but unknown in closed form. No stochastic expectation is considered, and we do not assume access to a distribution or confidence set for the cost vector. Instead, our challenge lies in the black-box nature of the lower level, where only oracle evaluations are available. This distinction makes our approach fundamentally different from stochastic approximation methods.

Black-box bilevel optimization. Finally, our contribution is situated within the broader black-box bilevel optimization literature, where the lower-level problem is accessible only through oracle evaluations and not through closed-form expressions. The work of Heitsch et al. [9] is particularly relevant in this regard, as they also study bilevel problems with convex black-box lower-level constraints. The key distinction, however, lies in our focus on a more restricted oracle model: in our framework, only function evaluations of the lower level are permitted, without access to gradient information. This setting makes classical cutting-plane constructions significantly more challenging and motivates the development of our specialized approach.

Finally, it is also necessary to mention that the robotic warehousing literature is vast and will not be detailed here, since our work, despite being motivated by these practical applications, aims to be applicable to broader settings.

3 Problem Formulation

3.1 Motivation

The motivation for our formulation comes from the idea of adding penalty terms to a given problem in order to enable consequence-aware decision making. For instance, in the context of warehousing operations, adding penalty terms for shelf assignments that are likely to increase congestion leads to more effective practical decisions when considering the total runtime. However, the exact weight to assign to these penalty terms is unknown and is often chosen heuristically through a few experiments. The purpose of our bilevel formulation is to transform this intuition into a more systematic framework. Starting from a given problem (referred to as the first-stage problem), the penalty terms can be treated as new variables that must satisfy coupling constraints with the first-stage variables, but whose contributions to the total objective are not directly accessible. This motivates the introduction of a second-stage linear program with an unknown cost vector, extending the simple one-stage formulation. From a higher-level perspective, this can be viewed as approximating the second stage with linear features (namely, the penalty terms), whose contribution to the bilevel program is unknown, and then solving the first-stage problem jointly with this approximation of the second stage.

3.2 Formulation

The problem is formalized as below :

$$\begin{aligned} &\min_{x\in\mathcal{X},\, y\in\mathcal{Y}(x)} \quad c^\top x + q_\dagger^\top y \\ &\text{where} \quad \begin{cases} \mathcal{X} := \{x \in \mathbb{R}^n \mid Ax = b,\, x \geq 0\} \\ \mathcal{Y}(x) := \{y \in \mathbb{R}^m \mid Wy = h - Tx,\, y \geq 0\} \end{cases} \end{aligned} \tag{1}$$

and $A \in \mathbb{R}^{l\times n},\ b \in \mathbb{R}^l,\ W \in \mathbb{R}^{p\times m},\ h \in \mathbb{R}^p,\ T \in \mathbb{R}^{p\times n}$ and $c \in \mathbb{R}^n$. x corresponds to the first-stage decision and y to the second stage decision. $\mathcal{X}$ corresponds to the first-stage constraints and $\mathcal{Y}(x)$ to the second-stage constraints, which are coupling constraints.

$q_\dagger \in \mathbb{R}^m$ is a completely **unknown** vector that aggregates all information about the downstream processes, which are represented by the second-stage variable y.

We will often refer to the recourse function defined as $Q(x) := \min_{y\in\mathcal{Y}(x)} q_\dagger^\top y$. $Q(\cdot)$ is the solution of the second-stage problem (SP_x) associated with the first-stage decision x. This allows to rewrite the problem in the more compact form :

$$\min_{x\in\mathcal{X}} c^\top x + Q(x) \tag{2}$$

To complete the problem formulation, a few simple assumptions are introduced to ensure a rigorous setting.

Assumption 1. *For every x in $\mathcal{X}$, the second-stage problem (SP_x) has a finite optimal solution.*

A standard assumption in two-stage problems is that, for every $x \in \mathcal{X}$, the second-stage problem (SP_x) admits a finite optimal solution. Indeed, it is well known that the set K of x such that SP_x admits a finite optimal solution is a convex polyhedron [6]. Hence the assumption is equivalent to replacing $\mathcal{X}$ with $\mathcal{X} \cap K$, which does not alter the problem's solution. We also note that $Q(\cdot)$ is a real-valued function, as a consequence of this assumption.

The oracle $\mathcal{O}_{q_\dagger}$ is defined as :

$$\begin{aligned} \mathcal{O}_{q_\dagger} : \mathcal{X} &\to \mathbb{R}^m \times \mathbb{R} \\ x &\mapsto (y^*(x), Q(x)) \end{aligned} \tag{3}$$

where $y^*(x)$ designates a solution of (SP_x).

Assumption 2. *The oracle $\mathcal{O}_{q_\dagger}$ is well-defined as a function.*

As stated above, for purposes of readability and clarity, we assume that $y^*(\cdot)$ is well-defined as a function. This means that it always returns the same value $y^*(x)$ for a given x, even when the second-stage problem admits multiple optima. This is not necessary for the sake of our solution, but rather a notational convenience to facilitate the presentation of our solution and proofs.

Assumption 3. *T is not the null matrix.*

If $T = 0_{p\times n}$, the problem becomes separable into two independent linear programs, which is not the purpose of our work. This case would be trivially solved by solving each part separately, the first one with any lp solver and the second by calling the oracle once.

Assumption 4. $\mathrm{Aff}(\mathcal{X}) = \{x \in \mathbb{R}^n \mid Ax = b\}$, *i.e. the equations* $Ax = b$ *define exactly the affine hull of* $\mathcal{X}$.

This assumption is made without loss of generality. Indeed, if instead $\mathrm{Aff}(\mathcal{X}) = \{x \in \mathbb{R}^n \mid Cx = d\}$, one simply replaces (C, d) by (A, b) without changing $\mathcal{X}$. Equivalently, no coordinate of x is identically zero on $\mathcal{X}$, and hence $\dim(\mathcal{X}) = \dim\big(\ker(A)\big)$. In the following work, we will denote $S := \mathrm{Aff}(\mathcal{X})$.

4 Proposed method

First, it should be emphasized that the proposed method relies on a decomposition between first-stage and second-stage decisions, although this structure is not intrinsic to the problem formulation itself. The decomposition is a design choice of our solution approach and alternative methods could address the problem without explicitly introducing such a separation. The rationale for adopting this decomposition lies in its ability to clearly isolate the black-box component from the part that admits a closed-form representation, a distinction that is crucial for effectively solving the problem. An alternative perspective is briefly discussed in Appendix 6.2, where the problem is reformulated as a robust optimization problem on $q_\dagger$.

The proposed approach can be interpreted as an instance of Kelley's cutting-plane algorithm [11] applied to the convex recourse function $Q(x)$. In the classical Benders decomposition [2] and its generalization to convex problems [8], subgradients are obtained from the dual solutions of the second-stage problem. In the present setting, subgradients are instead estimated from oracle evaluations, which enables the treatment of black-box recourse functions.

The main challenge is the generation of valid cuts without access to dual solutions of the second-stage problems. This is addressed by reconstructing the affine profile of the recourse function in the neighborhood of a given first-stage solution x. Oracle queries are performed at carefully selected neighboring points to compute directional derivatives. These points are centered around an anchor point located strictly within the affine region of the recourse function. A high-level description of the method is provided at Algorithm 1, with technical details deferred to subsequent subsections.

Algorithm 1 Decomposition method

initialize: $t \leftarrow 1$, (MP) : $\min\limits_{\substack{x \in \mathcal{X} \\ Q \geq 0}} c^\top x + Q$, LB $\leftarrow -\infty$ and UB $\leftarrow +\infty$
loop
  Solve MP and obtain solution x_t, LB $\leftarrow$ objective
  Query the oracle around x_t and build a local gradient ∇_B
  Generate a new cut for MP : $Q(x) \geq Q(x^\circ) + \nabla_B(x - x^\circ)$
  if UB − LB $<$ gap **then**
    break
  else
    UB $\leftarrow \max(\mathrm{UB}, c^\top x_t + Q(x_t))$
    $t \leftarrow t + 1$
  end if
end loop
return x_t

The following subsections respectively address the identification of affine regions of the recourse function, the computation of an anchor point for a given $x \in \mathcal{X}$, and the generation of a cut through

oracle queries.

4.1 Critical regions and recourse function

First, we describe rigorously the profile of the recourse function and identify the portions on which it is affine. We focus on the second-stage problem (SP_x), for any x in $\mathcal{X}$. Recall that it is written as :

$$\min_{\substack{Wy=h-Tx,\\ y\geq 0}} q_\dagger^\top y \tag{4}$$

If $I \subset \{1,\dots,m\}$, we denote by $W_I \in \mathbb{R}^{p\times|I|}$ the submatrix of W consisting of the columns indexed by I, and by $y_I \in \mathbb{R}^{|I|}$ the subvector of y with components indexed by I.

A basis $B \subset \{1,\dots,m\}$ is any subset of cardinality p such that W_B is invertible. We then set $N := \{1,\dots,m\} \setminus B$. Notice that there is a finite number of basis.

Definition 1. *[Basic points and optimal basis]*

(i) Let (B,N) be a partition of $\{1,\dots,m\}$ such that B is a basis. The associated basic point *is the unique vector $y \in \mathbb{R}^m$ with components*

$$y_B \;=\; W_B^{-1}(h-Tx), \qquad y_N \;=\; 0.$$

Notice that $Wy = h - Tx$.

(ii) B is primal feasible *(resp.* non-degenerate*)* at x *if*

$$W_B^{-1}(h-Tx) \;\geq\; 0 \quad \big(\textit{resp.} \;>0\big).$$

(iii) B is dual feasible *if its reduced costs are non-negative, i.e.*

$$q_{\dagger,N} \;-\; W_N^\top (W_B^{-1})^\top q_{\dagger,B} \;\geq\; 0$$

Notice this condition is independent of x.

(iv) B is optimal at x *if it is both primal feasible at x and dual feasible. More generally, B is called* optimal *if there exists some x in $\mathcal{X}$ for which it is optimal.*

Remark 1. For purpose of notation and to avoid any ambiguity in the following work, we introduce a *selection operator* S_B defined as follows :

$$\begin{aligned} S_B : \mathbb{R}^m &\to \mathbb{R}^p \\ x &\mapsto (x_i)_{i\in B} \end{aligned}$$

Notice that $S_B^\top : x \mapsto (x_i\,\mathbb{I}\{i\in B\})_{i=1}^n$ and $S_B S_B^\top = I_p$, $S_B^\top S_B = I_m$. More importantly, the basic point for the basis B is written :

$$y := S_B^\top W_B^{-1}(h-Tx)$$

Definition 2. *[Critical region]*

Let B be an optimal basis. The critical region CR_B *is defined as the set of x such that B is an optimal basis at x. We have :*

$$\mathrm{CR}_B = \{x \in \mathcal{X} \mid W_B^{-1}(h-Tx) \geq 0\} \tag{5}$$

Notice that critical regions are convex closed polyhedrons.

Remark 2. The concept of critical regions has been extensively studied in the multi-parametric programming literature, with a foundational contribution given in [7]. Critical regions also appear in the case where the entire right-hand side (RHS) is treated as a free parameter, hereafter referred to as the RHS setting. This is a more general framework than the one considered in this work, where the RHS is restricted to be an affine function of the parameter.

To describe our setting, it is useful to distinguish between three vector spaces:

1. **RHS space**: $\mathbb{R}^p$, where the right-hand side of the constraints lives.
2. **Affine RHS subspace**: $\text{Aff}(h, T) = \{h - Tx \mid x \in \mathbb{R}^n\} \in \mathbb{R}^m$ an affine subspace of dimension $\text{rank}(T)$ that contains all possible RHS vectors in our problem.
3. **x-space**: $\mathbb{R}^n$, where the parameter vector x lives.

In the RHS setting, critical regions are pointed, full-dimensional cones in the RHS space [12]. In our affine setting each critical region in the RHS space is obtained by intersecting such a cone with the affine subset $\{h - Tx \mid x \in \mathcal{X}\}$. This intersection produces critical regions in the affine subspace that are generally no longer full-dimensional. In some cases, the intersection may cause regions to collapse into lower-dimensional polyhedral sets in the x-space; for example, when $\text{Aff}(h, T)$ is tangent to an edge of a full-dimensional cone in the RHS space. Finally, the preimage of these affine-space regions under the transformation $x \mapsto h - Tx$ yields the critical regions in the x-space. This pullback preserves the level of degeneracy of each region.



(a) Critical regions in the RHS space.

(b) Critical regions in the x-space.

Figure 3: Critical regions represented in different vector spaces. [2]

Proposition 1. *[Coverage property]*

Let $\mathcal{B}$ be the collection of all optimal bases. The critical regions CR_B induce a coverage of the polyhedral region $\mathcal{X}$, formally

$$\mathcal{X} = \bigcup_{B \in \mathcal{B}} \text{CR}_B \tag{6}$$

[2] The parameter vector x is two-dimensional and the right-hand side vector is three-dimensional. The parameters are $h = (0, 0, 1)$ and $T = I_2$.

Proposition 2. *[Profile of the recourse function]*

The recourse function Q is piecewise linear convex. More precisely, on every critical region CR_B*, Q is affine and can be written as :*

$$Q_{|\mathrm{CR}_B} : x \mapsto q_\dagger^\top S_B^\top W_B^{-1}(h - Tx) \tag{7}$$

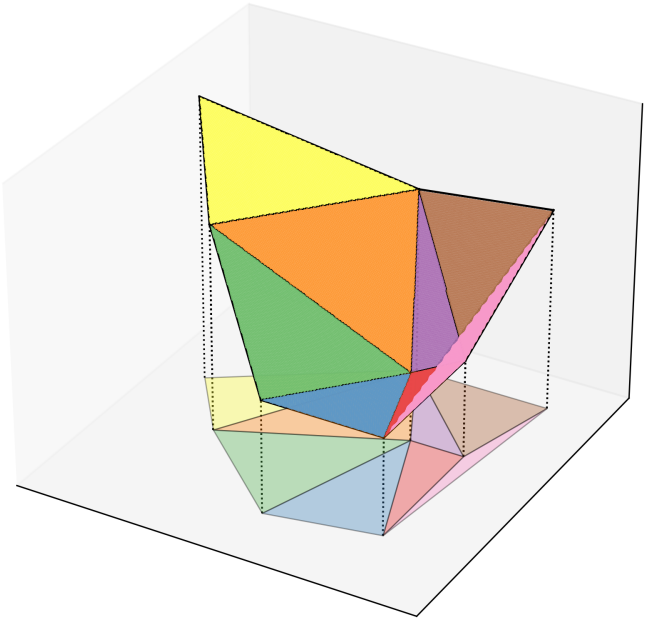


Figure 4: Visual of the recourse function with a 2d parameter

The piecewise-linear structure of the recourse function is key to constructing a lower approximation via Benders-type cuts. With the critical regions identified, the next subsection uses this result to define an anchor point for computing directional derivatives and reconstructing the affine profile around a given point x. The existence of this anchor point follows from the polyhedral coverage property.

4.2 Anchor point

Recall that S designates the affine hull of $\mathcal{X}$. $\mathrm{int}_S(\cdot)$ denotes the interior relative to the affine subspace $S = \{x \in \mathbb{R}^n \mid Ax = b\}$. The notion of an anchor point is formally defined below.

Definition 3. *[Anchor point]*

Let CR_B *be a critical region, and* $x \in \mathcal{X}$*.*

A core representative *of the critical region* CR_B *is a point in the interior of the critical region relative to S, namely* $x^\circ \in \mathrm{int}_S(\mathrm{CR}_B)$*.*

An anchor point *for x is any core representative of a critical region containing x.*

It follows from the definition that only full dimensional critical regions in S have core representatives. The algorithm will require to find an anchor point for a given x_t at each iteration t. The following result guarantees that such a point exists.

Theorem 1. *Let $x \in \mathcal{X}$. There exists a critical region CR_B such that :*

$$\begin{cases} x \in \mathrm{CR}_B \\ \mathrm{int}_S(\mathrm{CR}_B) \neq \emptyset \end{cases}$$

Hence, there always exists an anchor point for $x \in \mathcal{X}$.

Notice that the proof we provide for this result does not yield any constructive method for finding such an anchor point. The following subsections will develop a subroutine that systematically constructs an anchor point for a given $x \in \mathcal{X}$.

First, it is essential to identify a region that contain x. This can be done by querying the oracle and examining the support set of the optimal second-stage decision $y^*(x)$. Indeed, this support set is always contained within the set of basic indices at x, and moreover, it coincides with the basis indices whenever the subproblem solution is non-degenerate. The following subroutine details how to recover a critical region CR_B containing a given point $x \in \mathcal{X}$.

Algorithm 2 Basis identification subroutine

Require: Oracle query at x : $(y^*(x), Q(x)) \leftarrow \mathcal{O}_{q_\dagger}(x)$.
1: Let $B \leftarrow \mathrm{supp}(y^*(x))$.
2: **if** $|B| < p$ **then**
3: Find $W_{i_1}, \ldots, W_{i_{p-|B|}}$ such that $W_{B \cup \{i_1, \ldots, i_{p-|B|}\}}$ is full-rank.
4: Set $B \leftarrow B \cup \{i_1, \ldots, i_{p-|B|}\}$.
5: **end if**
6: **return** B

For the remainder of this work, we introduce the notation $\|y\|_0 := |\{i \in [\![1, m]\!] \mid y_i > 0\}|$ to denote the cardinality of the support set of a given non-negative vector y. We will use $\mathrm{supp}(\cdot)$ to designate the support set of vectors.

We distinguish between the cases where the second-stage is degenerate or non-degenerate, since it improves significantly the efficiency of the subroutine.

4.2.1 Non-degenerate case

We assume that $x \in \mathcal{X}$ is given such that the associated second-stage problem (SP_x) is non-degenerate. In particular, the optimal solution $y^*(x)$ is a non-degenerate basic point of the second stage polyhedron, meaning that $y^*(x) = W_B^{-1}(h - Tx) > 0$. This also implies that $B = \|y^*(x)\|_0$.

We also recall that the relative interior of $\mathcal{X}$ is not empty, allowing to fix $x^{\mathrm{int}} \in \mathrm{int}_S(\mathcal{X})$.

Lemma 1. *[Guarantee of interior]*

Let B be an optimal basis. If B is primal non-degenerate at a positive x, then x is in the interior of CR_B relative to S. Moreover, we have :

$$\mathcal{B}_{\mathcal{X}}(x, r) \subset \mathrm{CR}_B \quad \textit{where} \quad r = \min\left(\min_{1 \leq i \leq n} x_i, \frac{\min\limits_{1 \leq j \leq p} W_B^{-1}(h - Tx)}{\|W_B^{-1} T\|} \right)$$

By the preceding lemma, the only obstacle to x being an anchor point for itself is the possibility that x does not lie in the relative interior of the first-stage polyhedron $\mathcal{X}$, namely x has some zero coordinates. An anchor point associated with x can therefore be obtained by perturbing x along any strictly admissible direction. Since any such direction is valid, a convenient choice is to move toward a fixed interior reference point, independent of x, namely x^{int}. This observation is formalized in the following proposition.

Proposition 3. *[Anchor point]*

Let $x \in \mathrm{CR}_B \setminus \{x^{int}\}$ such that $W_B^{-1}(h - Tx) > 0$. We write $\delta := \min_{1 \leq i \leq m} \left(W_B^{-1}(h - Tx)\right)_i > 0$. Then :

$$x^\circ := x + r \cdot \frac{x^{int} - x}{\|x^{int} - x\|} \in \mathrm{int}_S(\mathrm{CR}_B) \quad \textit{with} \quad r := \frac{1}{2} \min\left(\frac{\delta}{\|W_B^{-1}T\|}, 1\right) > 0 \tag{8}$$

Therefore x° is an anchor point for x. Moreover, $\mathcal{B}(x^\circ, r) \subset \mathrm{CR}_B$.

In practice, we move along this direction using a fixed step size that is sufficiently small for all practical applications.

4.2.2 Degenerate case

In the degenerate case, some basic components of the second-stage decision have a zero coefficient, which makes the previous construction impossible. However, we still know from the proof of Theorem 1 that there exists a ball around x that is covered by regions containing x. Therefore, we can argue that if we make a step in a random direction, we will almost surely land on an anchor point for x. We formalize this intuition in the following proposition.

We denote as $\mathcal{S}(0, 1)$ the unit sphere in $\mathbb{R}^n$.

Proposition 4. *[Anchor point]*

Let $x \in \mathcal{X}$. Fix $\epsilon > 0$ like in the proof of Theorem 1, verifying equation 12. Consider the cone of strictly admissible directions at x, namely

$$C := \{d \in \mathbb{R}^n \mid Ad = 0,\ d_i > 0 \textit{ for every } i \textit{ st. } x_i = 0\}$$

Let d be randomly sampled from $C \cap \mathcal{S}(0, 1)$. Then, with probability 1, $x + \epsilon \cdot d \in \mathrm{int}_S(\mathrm{CR}_B)$ with $x \in \mathrm{CR}_B$. In this case, $x + \epsilon \cdot d$ is an anchor point for x.

4.2.3 Anchor point subroutine

The previous results are aggregated in the following subroutine, which systematically builds an anchor point for a given point $x \in \mathcal{X}$.

Remark 3. Notice that in the non-degenerate setting, both Lemma 1 and Proposition 3 provide a radius guarantee around the chosen anchor point. This radius guarantee can be efficiently used in the rest of the algorithm.

4.3 Generating a cut

In this section, we consider a point $x \in \mathrm{CR}_B$ together with an anchor point x°. Fix $\epsilon > 0$ such that $\mathcal{B}(x^\circ, \epsilon) \subset \mathrm{CR}_B$. ϵ is a radius guarantee around the anchor point. Fix a basis $(n_i)_{1 \leq i \leq r} \in (\mathbb{R}^n)^r$ of $\ker(\mathrm{A}) =: \mathcal{X}^\angle$, the subspace of feasible directions of the polyhedron. r is the dimension of $\ker(\mathrm{A})$.

Algorithm 3 Anchor point subroutine

Require: $x^{\text{int}} \in \text{int}_S(\mathcal{X})$ and $\epsilon > 0$.

Query the oracle at x : $(y^*(x), Q(x)) \leftarrow \mathcal{O}_{q_\dagger}(x)$
if $\|y^*(x)\|_0 = p$ **then**
  if $x > 0$ **then**
    return x
  else
    return $x^\circ := x + r \cdot \frac{x^{\text{int}} - x}{\|x^{\text{int}} - x\|}$ with r chosen like in Proposition 3
  end if
else
  Generate a random strictly admissible direction d
  return $x + \epsilon \cdot d$
end if

The role of the anchor point is to serve as a reference from which we can query the oracle along directions in $\mathcal{X}^\angle$, while remaining inside the critical region CR_B. This enables us to recover the affine profile of the recourse function over CR_B. The following proposition formalizes this idea.

Proposition 5. *[Cut validity]*
Let B be an optimal basis. Let $x^\circ \in \text{int}_S(\text{CR}_B)$.
Define

$$N := \begin{pmatrix} n_1 & \cdots & n_r \end{pmatrix} \in \mathbb{R}^{n \times r} \quad \textit{and} \quad g_B := \left(\frac{Q(x^\circ + \epsilon \cdot n_i) - Q(x^\circ)}{\epsilon} \right)_{i=1}^{r}$$

The cut

$$l_B : x \in \mathcal{X} \mapsto Q(x^\circ) + \nabla_B^\top (x - x^\circ) \leq Q(x) \quad \textit{where} \quad \nabla_B = N(N^\top N)^{-1} g_B \tag{9}$$

is a valid cut. Moreover, we have $l_{B|\text{CR}_B} = Q_{|\text{CR}_B}$.

It follows directly from the observation that l_B and Q coincide on CR_B, which is a full dimensional region, that the cut defined above is Pareto-optimal.

Proposition 6. *[Pareto-optimality]*
The cut defined above is always a Pareto-optimal cut.

This implementation is entirely correct, but it does not fully exploit the specific structure of our problem. In fact, the method applies to any piecewise-linear convex recourse function, provided that anchor points can be readily identified.

However, by leveraging our knowledge of the second-stage problem, we can achieve significantly greater efficiency in terms of oracle calls.

Remark 4. It is important to note that the objective function is linear with respect to the second stage optimal decision $y^*(x)$. More precisely, given a set of queries $(x_k)_{k \in K}$ with corresponding second stage solutions $(y_k)_{k \in K}$, we have, for any $x \in \mathcal{X}$:

$$\forall (\lambda_k)_{k \in K} \in \mathbb{R}^K, y^*(x) = \sum_{k \in K} \lambda_k y_k \implies Q(x) = \sum_{k \in K} \lambda_k Q(x_k)$$

Since, within any given critical region CR_B, we have access to a closed-form expression for $y^*(x)$ that depends only on B, W, h and T, we can leverage this linearity property to reduce the number of oracles calls. In particular, the oracle needs to be called only when the second-stage optimal decision for a given x does not lie in the span of the optimal decisions corresponding to previous query points at which the oracle was called.

To identify points x where the oracle needs not to be called, it will be necessary to check the

Remark 5. Another important observation is that the matrix T may be rank-deficient. As a consequence, the same linear dependencies between second-stage decisions occur across all critical regions, due to this rank deficiency of T.

To exploit this structure, consider the family of vectors $(v_i) := (Tn_i)_{i=1}^r \in \mathbb{R}^{p\times r}$. From (v_i) we extract a maximal linearly independent subfamily. Without loss of generality, we may reorder the vectors so that $(v_i)_{1\le i\le s}$ forms a basis of $\mathrm{span}\{(v_i)_{1\le i\le r}\}$. For each $i \in [\![s+1, r]\!]$, we fix coefficients $(\lambda_j^{(i)})_{1\le j\le s}$ such that :

$$v_i = \sum_{1\le j\le s} \lambda_j^{(i)} v_j$$

These linear combinations can be used to infer the value of the objective function at specific points and therefore avoid calling the oracle.

Proposition 7. *[Construction of g_B]*

Let CR_B be a critical region. Let $(x_k)_{k\in K}$ be a set of first stage decisions for which we know the oracle answer $(y_k, Q(x_k))_{k\in K}$. We assume that $x^\circ \in (x_k)_{k\in K}$ and we fix $y^\circ \in \mathbb{R}^m$ the associated optimal second stage decision. The following procedure builds g_B correctly.

Algorithm 4 Constructing g_B while limiting Oracle Calls

Require: For all $i \in [\![s+1, r]\!]$, precompute $(\lambda_j^{(i)})_{j=1}^s$ such that $Tn_i = \sum_{j=1}^s \lambda_j^{(i)} Tn_j \quad (\star)$
Ensure: $g_B := (g_{B,1}, \dots, g_{B,r})$

Build basis B using Algorithm 2 and the oracle query at x°
for $i = 1$ to s **do**
    Let $v \leftarrow S_B^\top W_B^{-1} Tn_i$.
    if $v \in \mathrm{span}\{(y_k)_{k\in K}\}$ **then**
        Find coefficients $\{\lambda_k^{(i)}\}_{k\in K}$ such that $v = \sum_{k\in K} \lambda_k^{(i)} y_k$.
        Set $g_{B,i} \leftarrow -\sum_{k\in K} \lambda_k^{(i)} Q(x_k)$
    else
        Query the oracle at $x^\circ + \epsilon \cdot n_i$ to obtain $(y_{\mathrm{new}}, Q_{\mathrm{new}})$.
        Add the new index to K and store y_{new}, Q_{new}.
        Set $g_{B,i} \leftarrow \frac{1}{\epsilon}(Q_{\mathrm{new}} - Q(x^\circ))$.
    end if
end for
for $i = s+1$ to r **do** ▷ Anticipated Linear Dependencies
    Set $g_{B,i} \leftarrow \sum_{j=1}^s \lambda_j^{(i)} g_{B,j}$.
end for
return g_B

Remark 6. It follows from the algorithm that the collection of oracle queries must be updated each time the oracle is called. Consequently, whenever the oracle is queried (specifically in Algorithm 2 at line 0, in Algorithm 3 at line 1, and in Algorithm 5 at line 7), we assume that a check is performed to determine whether the obtained optimal solution lies in the span of the current collection $(y_k)_{k\in K}$. If it does not, the oracle result collection is updated accordingly.

Remark 7. It is worth noting that we do not emphasize how the radius guarantee ϵ is determined, particularly in cases where the anchor point is obtained through a random perturbation. This choice is made to simplify the presentation of the algorithms, although it can be addressed more rigorously in practice. In our implementation, we fix an arbitrary radius ϵ that is sufficiently small for most practical cases, and we verify the validity of each oracle query using the support set of the returned optimal solution. Specifically, if the support set of the oracle output y_{new} is contained in the basis indices B, then this ensures that $x^\circ + \epsilon \cdot n_i \in \mathrm{CR}_B$. Conversely, if the support set is not contained in B, this indicates that the chosen radius ϵ is too large to remain within the region CR_B. In such cases, ϵ is simply reduced until a valid query is obtained.

Remark 8. Another important point concerns the way the condition at line 4 is tested. This step can become a significant bottleneck in practice and must therefore be handled carefully. The most effective approach is to maintain a QR decomposition of the matrix Y_K (see Lemma 2 for the definition of Y_K) in order to test the span condition efficiently. This QR decomposition should be updated each time the oracle returns a linearly independent optimal solution, using modified Gram–Schmidt computations. Regular re-orthonormalization is required to mitigate the numerical instability inherent in successive Gram–Schmidt steps.

4.4 Final solution

The technical developments presented in the preceding subsections allow us to express the final solution in a concise form. This procedure is summarized in Algorithm 5.

Algorithm 5 Final solution

1: **initialize:** $t \leftarrow 1$, (MP) : $\min\limits_{\substack{x\in\mathcal{X}\\ Q\geq 0}} c^\top x + Q$, LB $\leftarrow -\infty$ and UB $\leftarrow +\infty$, $K = \emptyset$
2: **compute:** requirement $(\star)$ from Algorithm 4
3: **compute:** matrix N and projector $P_N := N(N^\top N)^{-1}$.
4: **loop**
5: Solve MP and obtain solution x_t, LB $\leftarrow$ objective
6: Obtain an anchor point x° for x_t using Algorithm 3
7: Query the oracle for x° : $(y^\circ, Q^\circ) \leftarrow \mathcal{O}_{q_\dagger}(x^\circ)$
8: Compute g_B using algorithm 4
9: Generate a new cut for MP : $Q(x) \geq Q(x^\circ) + P_N\, g_B\, (x - x^\circ)$
10: **if** UB − LB $<$ gap **then**
11: **break**
12: **else**
13: UB $\leftarrow \max(\text{UB}, c^\top x_t + Q(x_t))$
14: $t \leftarrow t + 1$
15: **end if**
16: **end loop**
17: **return** x_t

The convergence of the algorithm and its correctness follow from results on decomposition methods and the correctness of the subroutines 1 and 3. An interesting result concerns the number of oracles calls made to obtain the final solution.

Lemma 2. *[Number of oracle calls]*
Let $(y_k, Q_k)_{k\in K}$ be the collection of oracle queries done at a given time. Then the matrix Y_K has full row rank *where :*

$$Y_K := \begin{pmatrix} y_1 \\ \vdots \\ y_K \end{pmatrix} \in \mathbb{R}^{K\times m}$$

It follows from the algorithm that the number of oracle queries is bounded above by the number of iterations plus the rank of the matrix Y_K. Indeed, each oracle call at the anchor point contributes one query per iteration, while the queries in the g_B subroutine always increase the rank of Y_K. In practice, however, additional queries may be required, particularly in cases of degeneracy. These situations are discussed in detail in Remark 7.

Remark 9. We have a trivial bound that gives us $\text{rank}(Y_K) = K \leq m$. However we can further restrict this bound to be more precise.

Proposition 8. *[Upper bound on the number of oracle calls]*
Let $V := \text{span}\{h - Tx \mid x \in \mathcal{X}\}$. For any collection of queries $(y_k, Q_k)_{k\in K}$, we have the following upper bound :

$$\text{rank}(Y_K) \leq m - (\text{rank}(W) - \dim(\text{Im}(W) \cap V)) \tag{10}$$

5 Computational Experiments

The computational experiments were done on a facility location instance, which can be written as follows :

$$\begin{aligned}
\min \quad & \sum_{j=1}^{n} c_j x_j + \sum_{j=1}^{n}\sum_{i=1}^{m} t_{ij} y_{ij} + \sum_{i=1}^{m} u_i z_i \\
\text{s.t.} \quad & \left(\sum_{j=1}^{n} y_{ij}\right) + z_i = d_i && \forall\, i \in [m] \\
& \sum_{i=1}^{m} y_{ij} \leq C_j x_j && \forall\, j \in [n] \\
& \mathbf{y}, \mathbf{z} \geq 0,\ \mathbf{x} \in [0,1]^n
\end{aligned}$$

where:

$$\begin{aligned}
x_j &= \text{proportion of facility } j \text{ built} \\
y_{ij} &= \text{quantity transported from facility } j \text{ to customer } i \\
z_i &= \text{unmet demand from customer } i
\end{aligned}$$

Notice that the first-stage decision, which is usually restricted to be binary in the traditional formulation of the problem, is assumed to be continuous here. The second-stage cost vectors t and

u were supposed to be unknown. In practice, the oracle was implemented using fixed values t and u to which the algorithm did not have access. The algorithm was run for various values of m and n, with the parameters chosen from a random distribution. The results are collected in the table below :

Table 1: Experimental results for varying n and m.

n	m	Gap Error	Iterations	Naive Oracles	Optimized Oracles
5	10	0.00	3.4	20.3	12.4
5	20	0.00	3.5	21.3	13.5
5	30	0.00	3.4	20.6	13.5
5	40	0.00	3.5	21.4	14.2
5	50	0.00	3.5	21.1	14.3
10	50	0.00	14.2	184.1	60.3
10	60	0.00	15.7	206.9	65.7
10	70	0.01	15.9	202.8	68.3
10	80	0.00	17.5	223.1	72.2
10	90	0.01	16.8	209.7	73.3
10	100	0.01	18.3	231.4	77.9
20	50	0.03	93.4	2022.0	205.6
20	60	0.02	101.4	2193.2	223.0
20	70	0.02	106.4	2301.2	204.6
20	80	0.07	112.9	2438.3	262.6
20	90	0.01	125.6	2711.8	287.9
20	100	0.13	115.5	2478.7	294.9

All the values given in the table are computed as mean values over 100 different seeds.

Two main observations can be drawn from these experiments. First, the gap error is not always equal to 0%, which may appear surprising. This discrepancy arises partly from the deliberate omission of certain cut-validity checks in the implementation in order to accelerate computations. Additional errors are attributable to numerical instability, which can occur when critical regions become too small.

The second important result concerns the proportion of oracle calls saved by applying the algebraic checks described in Algorithm 4. The evolution of this proportion as the size of the first-stage instance increases is illustrated in Figure 5. The results clearly indicate that algebraic checks are highly effective in reducing the number of oracle calls as the first-stage problem grows. In fact, Proposition 8 ensures that the optimized number of oracle calls depends only on the size of the second-stage problem, and not on that of the first stage. This explains the observed decrease in the proportion of calls saved when the first-stage problem grows more rapidly than the second-stage problem.

6 Conclusion and Future Work

6.1 Comments on the solution

Several aspects of the proposed solution are not entirely satisfactory, starting with the introduction of critical regions. Critical regions are defined relative to a basis B, yet the oracle does not provide

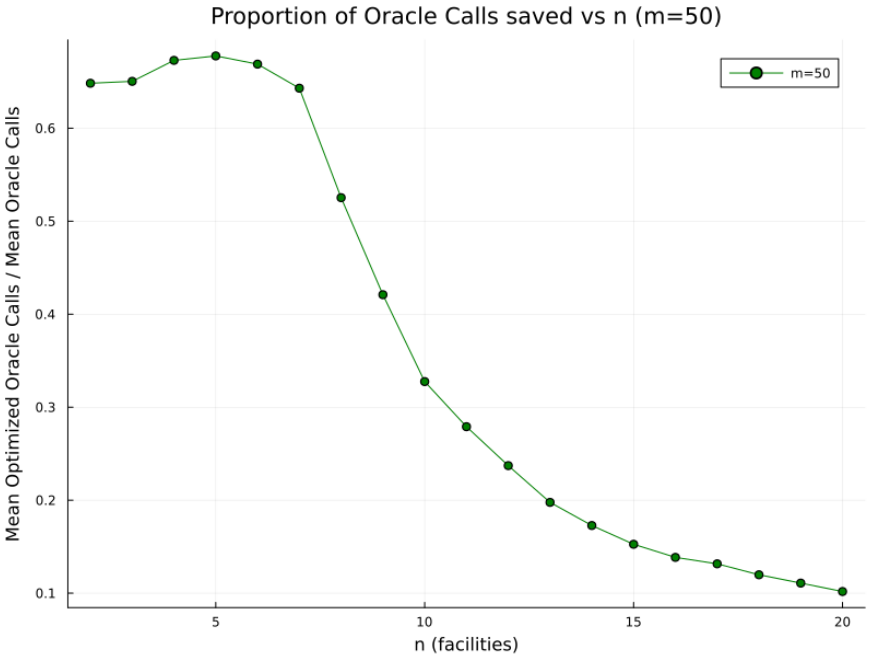


Figure 5: Proportion of oracle calls saved for different values of n

any basis information together with the solution of the second-stage problem. This basis is instead reconstructed by examining the support set of the solution. A more natural formulation might therefore be to introduce critical regions directly in terms of the support set rather than the optimal basis. These notions are thoroughly discussed in Hladík [10]. Nevertheless, our objective is to focus on full-dimensional regions, which are very likely to coincide with the critical regions defined in terms of the optimal basis, as considered in this work.

The methodology used to construct an anchor point in the presence of degeneracy relies on a probabilistic argument, which motivates the use of a random perturbation to identify a full-dimensional critical region. Although the argument is valid, it would be preferable to design a fully deterministic procedure.

Finally, it should be emphasized that the algebraic checks designed to reduce the number of oracle calls, such as the one at line 4 in Algorithm 4, can themselves be computationally expensive. While it is certainly valuable to limit the number of oracle queries, it must also be noted that these checks can impose a significant computational burden. Hence, this construction is particularly advantageous in settings where oracle calls are slow, for whatever reason.

6.2 Limitations and future work

Despite addressing the problem formalized in Section 3, several limitations of this work must be acknowledged. First, the proposed approach is not extendable to integer settings, as it relies fundamentally on continuity assumptions. This constitutes a serious limitation, since most practical problems are formulated as integer rather than linear programs, particularly in the context of robotic warehousing. Nevertheless, the algebraic checks used to limit oracle calls could, in principle, be applied in the integer setting and may be the only component of the method that transitions well. Further research is required to investigate how the ideas developed here could be adapted to integer formulations.

A second limitation also stems from the heavy reliance on continuity arguments, which raises

concerns regarding numerical stability. Numerical instability may pose a significant bottleneck for the algorithm. For instance, the procedure used to identify anchor points within critical regions is prone to instability when these regions become too small. Similar issues may arise in Subroutine 4, which constructs g_B, as briefly noted in Remark 8.

Finally, perhaps the most important limitation is the limited practical relevance of the exact setting studied here. In practice, it is unlikely that one would have access to an oracle returning both the solution and the objective value of the linear second-stage problem. Instead, it is more realistic to assume access to an oracle providing this information for the true second-stage process. For example, in a robotic warehousing application, one would typically observe the effective routing and the total runtime of operations for a given assignment of shelves. This suggests that a more relevant direction is the study of an **inexact** oracle, which returns values that do not exactly correspond to the linear proxy built from the second-stage variables y. It is hoped that the present work will serve as a first step toward addressing such inexact oracle formulations.

Acknowledgements

This work would not have been possible without the invaluable guidance and fruitful collaboration I received throughout my internship.

First and foremost, I would like to express my deep gratitude to Prof. Alexandre Jacquillat for his continuous and dedicated supervision. I could always rely on his attentive and supportive mentorship during the course of this project. I am also thankful for his exceptional teaching, which greatly enhanced my knowledge and provided me with all the necessary tools to address our research problem.

I would also like to warmly thank Felipe Cordera, with whom I collaborated on the first part of this work. I am particularly grateful for his help in formalizing the problem, but even more so for the numerous working sessions we shared. His efforts to set me up for success from the very beginning played a major role in making this internship an outstanding experience.

I wish to acknowledge the contributions of Prof. Cynthia Barnhart and Alexandria Schmidt, whose involvement made our meetings highly productive and greatly enriched this work.

Lastly, I am thankful to Prof. Stéphane Gaubert and Prof. Frédéric Meunier for their excellent teaching, which gave me a solid introduction to operations research, and to whose work I am indebted for providing a strong foundation for this report [4].

References

[1] Cynthia Barnhart, Alexandre Jacquillat, and Alexandria Schmid. *Robotic warehousing operations: a learn-then-optimize approach to large-scale neighborhood search*. 2024. arXiv: 2408.16890. URL: https://arxiv.org/abs/2408.16890.

[2] J. F. Benders. "Partitioning Procedures for Solving Mixed-Variables Programming Problems". In: *Numerische Mathematik* 4.1 (1962), pp. 238–252. DOI: 10.1007/BF01386316.

[3] Dimitri P. Bertsekas, Angelia Nedić, and Asuman Özdaglar. *Convex Analysis and Optimization*. Belmont, MA: Athena Scientific, 2003. ISBN: 1886529450.

[4] Frédéric Bonnans and Stéphane Gaubert. *Recherche Opérationnelle: Aspects Mathématiques et Applications*. Paris: Éditions de l'École Polytechnique, Mar. 2016. ISBN: 978-2-7302-1641-8.

[5] Boston Consulting Group. *Amplify Your Warehouse Automation ROI*. Insight Report. Warehouse automation service-level and cost ROI analysis. Boston Consulting Group, 2023. URL: https://www.bcg.com/publications/2023/amplify-warehouse-automation-roi.

[6] A. Charnes and W. W. Cooper. "Systems Evaluation and Repricing Theorems". In: *Management Science* 9 (1962), pp. 209–228.

[7] Tomas Gal and Josef Nedoma. "Multiparametric Linear Programming". In: *Management Science* 18.7 (1972), pp. 406–422.

[8] A. M. Geoffrion. "Generalized Benders Decomposition". In: *Journal of Optimization Theory and Applications* 10.4 (1972), pp. 237–260. DOI: 10.1007/BF00934810.

[9] Holger Heitsch et al. "On convex lower-level black-box constraints in bilevel optimization with an application to gas market models with chance constraints". In: *Journal of Global Optimization* 84.3 (2022), pp. 651–685. DOI: 10.1007/s10898-022-01161-z.

[10] Milan Hladík. "Multiparametric linear programming: Support set and optimal partition invariancy". In: *European Journal of Operational Research* 202.1 (2010), pp. 25–31. DOI: 10.1016/j.ejor.2009.04.019.

[11] J. E. Kelley. "The Cutting-Plane Method for Solving Convex Programs". In: *Journal of the Society for Industrial and Applied Mathematics* 8.4 (1960), pp. 703–712. DOI: 10.1137/0108053.

[12] Ilbin Lee, Stewart Curry, and Nicoleta Serban. "Solving Large Batches of Linear Programs". In: *INFORMS Journal on Computing* 31.2 (2019), pp. 302–317. DOI: 10.1287/ijoc.2018.0838.

[13] Efstratios N. Pistikopoulos et al. "Multiparametric programming in process systems engineering: Recent developments and path forward". In: *Computers & Chemical Engineering* 136 (2020), p. 106808. DOI: 10.1016/j.compchemeng.2020.106808.

[14] Alexander Shapiro, Darinka Dentcheva, and Andrzej Ruszczyński. *Lectures on Stochastic Programming: Modeling and Theory*. Vol. 9. MOS-SIAM Series on Optimization. SIAM, 2009. DOI: 10.1137/1.9780898718751.

[15] John Sherman and Winifred J. Morrison. "Adjustment of an Inverse Matrix Corresponding to Changes in the Elements of a Given Column or a Given Row of the Original Matrix". In: *Annals of Mathematical Statistics* 21.1 (1950), pp. 124–127.

Appendix : Notation Table

Symbol	Description
x	First-stage decision variable
$\mathcal{X}$	First-stage constraints $\{x \in \mathbb{R}^n \mid Ax = b,\, x \geq 0\}$
$\mathcal{X}^{\angle}$	Linear part of the affine subspace $\mathcal{X}$, namely $\mathrm{Ker}(A)$ of dimension r
y	Second-stage decision variable
$\mathcal{Y}(x)$	Second-stage constraints $\{y \in \mathbb{R}^m \mid Wy = h - Tx,\, y \geq 0\}$
$q_\dagger$	Unknown second-stage cost vector
$Q(\cdot)$	Recourse function $x \mapsto \min_{y \in \mathcal{Y}(x)} q_\dagger^\top y$
$\mathcal{O}_{q_\dagger}$	Oracle function $x \mapsto (y^*(x), Q(x))$
CR_B	Critical region associated with the basis B
S_B	Selection operator associated to the basis B
S	Affine subspace $\{x \in \mathbb{R}^n \mid Ax = b\}$. Affine hull of $\mathcal{X}$
int_S	Interior relative to the affine subspace S
x°	Anchor point
x^{int}	Fixed point in the relative interior of $\mathcal{X}$
$\|\cdot\|_0$	Cardinal of the support set of a given vector
$(n_i)_{1 \leq i \leq r}$	Orthonormal basis of the nullity space of A
g_B	Directional derivatives of Q in the critical region CR_B along the directions $(n_i)_{1 \leq i \leq r}$
∇_B	Gradient of Q in the critical region CR_B
$(y_k, Q_k)_{k \in K}$	Collection of oracle call results
$\mathcal{B}_{\mathcal{X}}(x, \epsilon)$	Open ball centered on x and of radius ϵ inside the polyhedron subspace $\mathcal{X}$

Table 2: Summary of Notation

Appendix : Exploring two neighboring basis

In this section, we examine in detail the exploration of two neighboring regions, and show that the number of oracle calls can be significantly reduced by exploiting structural properties of the recourse function. We first clarify the notion of neighboring. Two regions are *algebraic neighbors* if their bases differ by a single index swap, and *geometric neighbors* if they share a common facet in the ambient space. These notions are equivalent, as established in [12]. Henceforth, we simply write neighboring regions without further distinction.

Let B and B' be two set of indices differing by a single swap $i \leftrightarrow j$. We assume that the region B has already been explored, which means that we have computed the gradient : $\nabla_B = -q_\dagger^\top S_B^\top W_B^{-1} T$. The purpose of this section is to prove that **the gradient $\nabla_{B'}$ can be obtained with only one oracle call in the critical region $\mathbf{CR}_{B'}$.**

The key idea relies on the Sherman–Morrison formula [15], which expresses $W_{B'}^{-1}$ as a rank-one update of W_B^{-1}. We further establish a lemma showing that, when combined with their associated selection matrices, these inverses still differ by a rank-one term. This property implies that the corresponding optimal solutions $y^*(x)$ in the two neighboring regions are tightly related. To the

best of our knowledge, this result has not been explicitly stated in the existing literature.

Lemma 3. *$S_B^\top W_B^{-1} - S_{B'}^\top W_{B'}^{-1}$ is a matrix of rank 1.*

Proof. We can suppose that the entering variable is added at position i in the basis. Therefore, we can write :

$$W_{B'} = W_B + (W_j - W_i)e_i^\top$$

We can derive from Sherman-Morrisson formula [15] that :

$$W_{B'}^{-1} = W_B^{-1} - \underbrace{\frac{W_B^{-1}(W_j - W_i)e_i^\top W_B^{-1}}{1 + \langle e_i, W_B^{-1}(W_j - W_i)\rangle}}_{:=M \text{ of rank } 1} = W_B^{-1} - \frac{\overbrace{W_B^{-1}(W_j - W_i)}^{=:f\in\mathbb{R}^p}\ \overbrace{e_i^\top W_B^{-1}}^{=:g^\top\in\mathbb{R}^{1\times p}}}{\underbrace{e_i^\top W_B^{-1} W_j}_{=:\alpha\in\mathbb{R}}}$$

since $W_B^{-1}W_i = e_i$. We have the guarantee that $\alpha \neq 0$ because $W_{B'}$ is invertible as an optimal basis. We also know that :

$$S_{B'}^\top = S_B^\top + \underbrace{(\tilde{e}_j - \tilde{e}_i)e_i^\top}_{:=L}$$

where $(\tilde{e}_r)$ is the canonical basis of $\mathbb{R}^m$ and (e_s) the canonical basis of $\mathbb{R}^p$. Therefore, we obtain the result announced :

$$\begin{aligned}
S_B^\top W_B^{-1} - S_{B'}^\top W_{B'}^{-1} &= S_B^\top W_B^{-1} - (S_B^\top + L)(W_B^{-1} - M) \\
&= S_{B'}^\top M - L W_B^{-1} \\
&= \frac{1}{\alpha} S_{B'}^\top f g^\top - (\tilde{e}_j - \tilde{e}_i)e_i^\top W_B^{-1} \\
&= \underbrace{\left[\frac{1}{\alpha} S_{B'}^\top f - (\tilde{e}_j - \tilde{e}_i)\right]}_{=:\tilde{f}\in\mathbb{R}^m} g^\top \\
&= \tilde{f} g^\top \text{ of rank 1.}
\end{aligned}$$

□

Proposition 9. *Let $x \in CR_{B'} \setminus \mathrm{CR}_B$. We have :*

$$\nabla_{B'} = \nabla_B + \frac{l_B(x) - Q(x)}{g^\top(h - Tx)}\, g^\top T \tag{11}$$

This proves that one query in the neighboring region is sufficient to recover the gradient in this region, since g, h and T are known quantities.

Proof. Using the result from the Lemma 3, we can start by writing that :

$$\begin{aligned}
\nabla_{B'} &= -q_\dagger^\top S_{B'}^\top W_{B'}^{-1} T \\
&= -q_\dagger^\top (S_B^\top W_B^{-1} - \tilde{f} g^\top) T \\
&= \nabla_B + q_\dagger^\top \tilde{f} g^\top T
\end{aligned}$$

We can also rewrite the value of $Q(x)$ by using the Lemma 3 :

$$\begin{aligned} Q(x) &= q_\dagger^\top S_{B'}^\top W_{B'}^{-1}(h - Tx) && \text{because } x \in \mathrm{CR}_{B'} \\ &= q_\dagger^\top (S_B^\top W_B^{-1} - \tilde{f} g^\top)(h - Tx) && \text{using the lemma} \\ &= l_B(x) - q_\dagger^\top \tilde{f} g^\top (h - Tx) && \text{by definition of } l_B \end{aligned}$$

Notice that $x \notin \mathrm{CR}_B$, so we know that $Q(x) > l_B(x)$. This implies that $g^\top(h - Tx) \neq 0$, otherwise we would have $Q(x) = l_B(x)$. Therefore, we obtain that :

$$q_\dagger^\top \tilde{f} = \frac{l_B(x) - Q(x)}{g^\top (h - Tx)}$$

hence the result when combining with the expression above. □

Appendix : Interpretation in terms of information on $q_\dagger$ and robust problem

The intuition underlying the results on the number of oracle calls required can be further clarified by viewing the problem as one of progressively acquiring information about the unknown vector $q_\dagger$. Each oracle call provides two distinct types of information. First, we learn that $q_\dagger$ lies in the hyperplane

$$H_x := \left\{ q \mid q^T y^*(x) = Q(x) \right\}.$$

Second, we know that $q_\dagger$ belongs to the optimality cone

$$C_x := \left\{ q \mid \exists \pi,\ \pi^T W \leq q,\ \pi^T (h - Tx) = q^T y^*(x) \right\}$$

which is the set of cost vectors q that explain the optimality of $y^*(x)$ for the second-stage subproblem. Thus, after each oracle call at some x, we can refine our confidence set S of feasible candidates for $q_\dagger$. This set is updated as follows:

$$S \leftarrow S \cap C_x \cap H_x$$

This construction quantifies the information gathered about the vector $q_\dagger$. More importantly, it enables us to assess how this knowledge can be leveraged to improve our understanding of the recourse function. One natural way to measure the current information on the second-stage cost vector $q_\dagger$ is to consider the associated robust problem and subproblem:

$$(P^{(S)}) \quad \min_{x \in \mathcal{X}} c^\top x + \max_{q \in S} \min_{y \in \mathcal{Y}(x)} q^\top y \quad \text{and} \quad (SP_x^{(S)}) \quad \max_{q \in S} \min_{y \in \mathcal{Y}(x)} q^\top y.$$

Building on the formulation of the robust subproblem, we introduce the corresponding robust recourse function, defined as:

$$Q^{(S)} : x \mapsto \max_{q \in S} \min_{y \in \mathcal{Y}(x)} q^T y.$$

Corollary 1. *[Upper bound]*
The function $Q^{(S)}$ provides an upper bound on Q, that is: $\forall\, x \in \mathcal{X},\ Q(x) \leq Q^{(S)}(x)$.

Proof. The results follows directly from the fact that $q_\dagger \in S$. □

The following result characterizes precisely the effect of an oracle call on the robust problem. Intuitively, querying the oracle at a point x eliminates the uncertainty associated with the recourse value at that point: the robust recourse function then coincides with the true recourse function at x. More formally, if the hyperplane H_x is added to the set S as a result of an oracle call at x, then the two recourse functions agree pointwise at x.

Proposition 10. *[Strength of the hyperplane condition]*
Let (x, y, q) be a solution of $(P^{(S)})$. Then we have :

$$q \in H_x \implies \begin{cases} q \in C_x \\ Q^{(S)}(x) = Q(x) \end{cases}$$

Proof. The following diagram gives the necessary elements to understand what is lying under this result.

$$\begin{array}{ccc} & q^T y & \\ \swarrow^{(1)} & & \searrow^{(2)} \\ q^T y^*(x) & =^{(3)} & q_\dagger^T y^*(x) \end{array}$$

(1) comes from the optimality of y for the problem $\min_{y \in \mathcal{Y}(x)} q^T y$.

(2) comes from $Q(x) \leq Q^{(S)}(x)$

(3) comes from the assumption that $q \in H_x$.

We immediately see that the assumption forces $q^T y^*(x) = q^T y$, which means that $q \in S_x$. We also note that it forces $q^T y = q_\dagger^T y^*(x)$, which means $Q^{(S,H)}(x) = Q(x)$. □

This result shows that if the uncertainty set is defined as $S = \cap_{x \in \mathcal{X}} H_x$, then the solution to the robust problem coincides with that of the true problem. This is clear since calling the oracle for every $x \in \mathcal{X}$ amounts to fully mapping the unknown recourse function. The key intuition behind the need for only a limited number of oracle calls is that each set H_x is an affine hyperplane. Consequently, the intersection $\cap_{x \in \mathcal{X}} H_x$ can be expressed as a finite intersection. In particular, there exist m' first-stage solutions $x_1, \ldots, x_{m'}$ such that :

$$\bigcap_{x \in \mathcal{X}} H_x = \bigcap_{i=1}^{m'} H_{x_i}$$

where m' is bounded by the same quantity as in Proposition 8. Hence the need for only a limited number of oracle calls.

Appendix : Proof of results of Section 4

Proof of results of Subsection 4.1

Proof of Definition 2

Proof. This result is clearly stated in [7]. We regive the main arguments as part of this work.

Let B be an optimal basis. We know that B is dual feasible since this condition does not depend on x (cf. Definition 1). Therefore B is optimal at a point x if and only if the associated basic point is primal feasible, i.e. $W_B^{-1}(h - Tx) \geq 0$. One can rewrite : $\mathrm{CR}_B = \{x \in \mathcal{X} \mid W_B^{-1} T x \leq W_B^{-1} h\}$ which clearly shows that CR_B is a closed convex polyhedron, as the intersection of a closed convex polyhedron with a finite collection of closed half-spaces. □

Proof of Proposition 1

Proof. Again, this result is clearly stated in [7].

It is well-known that the solution set of a linear program always contains a basic point, provided it is not empty. Therefore, for every x in $\mathcal{X}$, there exists a basis B such that B is optimal at x, i.e. $x \in \mathrm{CR}_B$. Hence the result. □

Proof of Proposition 2

Proof. Again, this result is clearly stated in [7]. We re-give a proof as part of this work. Remind that $Q : x \mapsto \min_{y \in \mathcal{Y}(x)} q_\dagger^\top y$ is defined on the polyhedron $\mathcal{X}$, which is convex.

Let $x_1, x_2 \in \mathcal{X}$ and $0 < t < 1$. Let y_1, y_2 be some optimal solutions associated with the problems with x_1 and x_2 respectively. We can verify that $ty_1 + (1-t)y_2 \in \mathcal{Y}(tx_1 + (1-t)x_2)$ so $Q(tx_1 + (1-t)x_2) \leq q_\dagger^\top(ty_1 + (1-t)y_2) = tQ(x_1) + (1-t)Q(x_2)$, hence the convexity.

Then, for $x \in \mathrm{CR}_B$, we know that the basis B is optimal so the optimal objective is attained by the basis point $S_B^\top W_B^{-1}(h - Tx)$, hence the expression given. The piece-wise linear property follows from the coverage property stated in Proposition 1. □

Proof of results of Subsection 4.2

Proof of Theorem 1

Proof. Recall that we write $S := \mathrm{Aff}(\mathcal{X})$ the smallest affine subspace containing $\mathcal{X}$. Recall also that $\mathrm{int}_S(\cdot)$ refers to the interior relatively to that affine subspace $S \subset \mathbb{R}^n$. Finally, remind that $\mathcal{B}_\mathcal{X}(x, \epsilon)$ is the open ball centered on x and of radius ϵ inside the polyhedron $\mathcal{X}$, namely $\mathcal{B}(x, \epsilon) \cap \mathcal{X}$. We can write : $\mathcal{B}_\mathcal{X}(x, \epsilon) = \{x + \epsilon v \mid v \in \mathcal{X}^\angle, \|v\| < 1,\, x + \epsilon v \geq 0\}$.

Let $x \in \mathcal{X}$.

(i) Let $\epsilon > 0$. Recall from the coverage property stated in Proposition 1 that :

$$\mathcal{B}_\mathcal{X}(x, \epsilon) \subset \mathcal{X} = \bigcup_{B \in \mathcal{B}} \mathrm{CR}_B$$

The point of this first part is to prove that, provided we choose ϵ to be small enough, we can remove the critical regions that do not contain x from the coverage of the ball $\mathcal{B}_\mathcal{X}(x, \epsilon)$.

Let CR_B be a critical region such that $x \notin \mathrm{CR}_B$. We know from Definition 2 that CR_B is a closed convex set of $\mathbb{R}^n$, hence there is a linear functional z that strictly separates x from CR_B. By replacing z with $z/\|z\|$, we may assume that $\|z\| = 1$.

Denote $\epsilon_B := \inf_{y \in \mathrm{CR}_B}(z \cdot y) - z \cdot x > 0$ and let $v \in \mathcal{X}^\angle$ such that $\|v\| < 1$. We have :

$$\begin{aligned} z \cdot \big(x + \epsilon_B v\big) &\leq z \cdot x + \epsilon_B\, z \cdot v \\ &< z \cdot x + \epsilon_B \qquad \text{since } z \cdot v \leq \|z\|.\|v\| < 1 \text{ and } \epsilon_B > 0 \\ &< \inf_{y \in \mathrm{CR}_B} z \cdot y \qquad \text{by defintion of } \epsilon_B \end{aligned}$$

so $x + \epsilon_B v$ cannot belong to CR_B. Therefore, $\mathcal{B}(x, \epsilon_B) \cap \mathrm{CR}_B = \emptyset$.

Now consider $\epsilon := \min\big\{\epsilon_B \mid x \notin \mathrm{CR}_B\big\}$. $\epsilon > 0$ because there is a finite number of optimal basis. We obtain using the previous work that :

$$\mathcal{B}_\mathcal{X}(x, \epsilon) \subset \bigcup_{B \text{ st. } x \in \mathrm{CR}_B} \mathrm{CR}_B \tag{12}$$

(ii) Even though this is visually intuitive, we prove formally that $\text{int}_S(\mathcal{B}_\mathcal{X}(x,\epsilon)) \neq \emptyset$. We leverage that $\mathcal{X}$ is convex so $\overline{\text{int}_S(\mathcal{X})} =: \overline{\text{relint}(\mathcal{X})} = \overline{\mathcal{X}} \supseteq \mathcal{X}$. Therefore, we can find $x' \in \text{int}_S(\mathcal{X}) \cap \mathcal{B}_\mathcal{X}(x,\epsilon)$. Fix $\delta > 0$ such that $\mathcal{B}_S(x',\delta) \subset \mathcal{X}$. We also know that $\|x - x'\| < \epsilon$. Define $\delta' := \min(\delta, \epsilon - \|x - x'\|) > 0$. We want to show that $\mathcal{B}_S(x',\delta') \subset \mathcal{B}_\mathcal{X}(x,\epsilon)$, which will suffice to conclude since $\mathcal{B}_S(x',\delta')$ is open in S and non-empty.

First, $\mathcal{B}_S(x',\delta') \subset \mathcal{B}_S(x',\delta) \subset \mathcal{X}$ because $\delta' \leq \delta$. Then let $x'' \in \mathcal{B}_S(x',\delta')$. We have :

$$\|x - x''\| \leq \|x - x'\| + \underbrace{\|x' - x''\|}_{\leq \epsilon - \|x - x'\|} \leq \epsilon$$

hence the result.

(iii) By contradiction, assume that for every B such that $x \in \text{CR}_B$, we have $\text{int}_S(\text{CR}_B) = \emptyset$. Then by finite reunion, we would have

$$\text{int}_S\left(\bigcup_{B \text{ st. } x \in \text{CR}_B} \text{CR}_B \right) = \emptyset$$

which is not possible since $\mathcal{B}_\mathcal{X}(x,\epsilon)$ is a subset of the reunion (cf. **(i)**) and has non-empty interior in S (cf. **(ii)**). Thus there exists at least one basis B such that :

$$\begin{cases} x \in \text{CR}_B \\ \text{int}_S(\text{CR}_B) \neq \emptyset \end{cases}$$

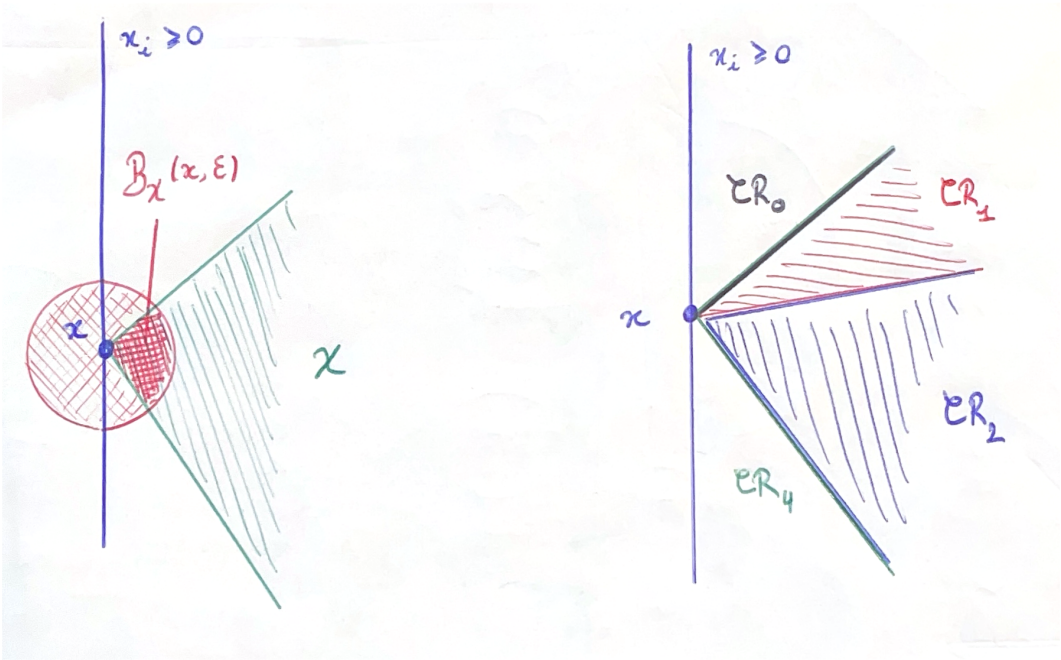


Figure 6: Illustration for part **(i)** and **(ii)** of the proof

□

Proof of Lemma 1

Proof. Recall the expression of CR_B given at Definition 2, namely

$$\mathrm{CR}_B = \{x \in \mathbb{R}^n \mid Ax = b,\, W_B^{-1}(h - Tx) \geq 0,\, x \geq 0\}.$$

Let r be defined like in the Lemma. Let $v \in \mathcal{X}^{\angle}$ such that $\|v\| \leq 1$. First we have for $i \in [\![1, n]\!]$:

$$\begin{aligned}(x + r \cdot v)_i &\geq x_i - r \cdot \|v\| \\ &\geq x_i - r \\ &\geq 0\end{aligned}$$

Then we can also write for $j \in [\![1, p]\!]$:

$$\begin{aligned}(W_B^{-1}(h - T(x + r \cdot v))_j &\geq (W_B^{-1}(h - Tx))_j - \|W_B^{-1}T\| \cdot r \cdot \|v\| \\ &\geq (W_B^{-1}(h - Tx))_j - \|W_B^{-1}T\| \cdot r \\ &\geq 0\end{aligned}$$

Hence $\mathcal{B}_{\mathcal{X}}(x, r) \subset \mathrm{CR}_B$. Consequently, $x \in \mathrm{int}_S(\mathrm{CR}_B)$. □

Proof of Proposition 3

Proof. Notice that r is well defined because $W_B^{-1}T$ is not null. Indeed, W_B is invertible and Assumption 3 ensures that T is not null.

Let $x \in \mathrm{CR}_B$ such that $W_B^{-1}(h - Tx) > 0$ and $x \neq x^{\mathrm{int}}$. Let $d := r \cdot \frac{x^{\mathrm{int}} - x}{\|x^{\mathrm{int}} - x\|}$. According to Lemma 1, it is sufficient to prove that $x + d$ is positive, lies in S and that $W_B^{-1}(h - T(x + d)) > 0$.

First, we know that $r \in]0, 1[$, so $x + d \in \mathrm{int}_S(\mathrm{CR}_B)$ by the line-segment principle [3]. From this we obtain that $x + d$ is positive and lies in S. Then we need to write :

$$\begin{aligned}\left(W_B^{-1}(h - T(x + d))\right)_i &= \left(W_B^{-1}(h - Tx)\right)_i - \left(W_B^{-1}Td\right)_i \\ &\geq \delta - \|W_B^{-1}Td\| \\ &\geq \delta - r\|W_B^{-1}T\| \\ &> 0\end{aligned}$$

Hence $x + d \in \mathrm{int}_S(\mathrm{CR}_B)$. The same type of computations prove that $\mathcal{B}_{\mathcal{X}}(x^{\circ}, r) \subset \mathrm{CR}_B$. □

Proof of Proposition 4

Proof. Recall that we have by assumption :

$$\mathcal{B}_{\mathcal{X}}(x, \epsilon) \subset \bigcup_{B \text{ st. } x \in \mathrm{CR}_B} \mathrm{CR}_B$$

This implies that :

$$D := \{x + \epsilon \cdot d \mid d \in C \cap \mathcal{S}(0, 1)\} \subset \bigcup_{B \text{ st. } x \in \mathrm{CR}_B} \mathrm{CR}_B$$

If we denote μ the Borel measure in $\mathbb{R}^n$, we have for every region CR_B containing x :

$$\mathbb{P}(x + \epsilon \cdot d \in \mathrm{int}_S(\mathrm{CR}_B)) = \frac{\mu(\mathrm{int}_S(\mathrm{CR}_B) \cap D)}{\mu(D)} = 0$$

for any degenerate region. Hence the result. □

Proof of results of Subsection 4.3

Proof of Proposition 5

Proof. Since Q is affine on the critical region CR_B, for all $x \in \mathrm{CR}_B$ we have $Q(x) = Q(x^\circ) + \nabla Q(x^\circ)^\top (x - x^\circ)$. By weak duality, the same linear form underestimates Q off CR_B, i.e.

$$Q(x) \geq Q(x^\circ) + \nabla Q(x^\circ)^\top (x - x^\circ) \quad \forall\, x \notin \mathrm{CR}_B.$$

Thus it suffices to check that $\nabla_B := N(N^\top N)^{-1} g_r \;=\; \nabla Q(x^\circ)$. But

$$g_r = \big(Q(x^\circ + n_i) - Q(x^\circ)\big)_{i=1}^r = \big(\nabla Q(x^\circ)^\top n_i\big)_{i=1}^r = N^\top \nabla Q(x^\circ),$$

By replacing $\nabla Q(x^\circ)$ by its projection on $\mathcal{X}^\angle$, we can assume without loss of generality that $\nabla Q(x^\circ) \in \mathcal{X}^\angle$. We write $\nabla Q(x^\circ) = Nz$. Then

$$N^\top N\, z = N^\top \nabla Q(x^\circ) = g_r \implies z = (N^\top N)^{-1} g_r,$$

hence

$$\nabla Q(x^\circ) = N\, z = N(N^\top N)^{-1} g_r = \nabla_B.$$

Note that it follows from $\nabla Q(x^\circ) = \nabla_B$ that l_B and Q coincide on CR_B. □

Proof of Proposition 6

Proof. Recall that the cut l_B is defined as $x \mapsto Q(x^\circ) + \nabla_B (x - x^\circ)$. Let $l' : x \mapsto Q(x^\circ) + b'^\top (x - x^\circ)$ be a valid cut such that $l'(x) \geq l_B(x)$ for every $x \in \mathcal{X}$.

Proposition 5 gives us that l_B and Q coincide on CR_B. Necessarily l' coincides with Q on CR_B by bounding. Since l_B and l' are affine and coincide on an open set in S, we have that $l_B = l'$ on S. Hence l_B is necessarily Pareto optimal. □

Proof of Proposition 7

Proof. Let CR_B be a critical region. Recall that g_B is defined as :

$$g_B := \left(\frac{Q(x^\circ + \epsilon \cdot n_i - Q(x^\circ))}{\epsilon}\right)_{i=1}^r = \left(-q_\dagger S_B^\top W_B^{-1} T n_i\right)_{i=1}^r$$

using the expression of Q on the critical region CR_B detailed in Proposition 2. We want to prove that the Algorithm 4 correctly computes every component of g_B.

First, let $i \in [\![1, s]\!]$. If the condition at line 4 is true, we can fix $(\lambda_k^{(i)})_{k \in K}$ such that

$$S_B^\top W_B^{-1} T n_i := v = \sum_{k \in K} \lambda_k^{(i)} y_k$$

Then, we obtain by linearity :

$$\begin{aligned} g_{B,i} &:= -\sum_{k \in K} \lambda_k^{(i)} Q(x_k) \\ &= -\sum_{k \in K} \lambda_k^{(i)} q_\dagger^\top y_k \\ &= -q_\dagger^\top v \end{aligned}$$

Hence the result for $i \in [\![1, s]\!]$. Now we consider $i \in [\![s+1, r]\!]$. Using the requirement $(\star)$, we can fix $(\lambda_j^{(i)})_{j=1}^s \in \mathbb{R}^s$ such that $Tn_i = \sum_{j=1}^s \lambda_j^{(i)} Tn_j$. We obtain directly by linearity that

$$\begin{aligned} g_{B,i} &:= \sum_{j=1}^{s} \lambda_j^{(i)} g_{B,j} \\ &= \sum_{j=1}^{s} \lambda_j^{(i)} (-q_\dagger S_B^\top W_B^{-1} T n_j) \qquad \text{using the results above} \\ &= -q_\dagger S_B^\top W_B^{-1} T n_i \qquad\qquad\qquad\qquad\qquad\qquad \text{by linearity} \end{aligned}$$

Hence the result for every component of g_B. □

Appendix : Proof of results of Subsection 4.4

Proof of Lemma 2

Proof. From the formulation of the different algorithms, we obtain that every time an oracle result $(y^*(x), Q(x))$ is added to the collection $(y_k, Q_k)_{k\in K}$, the optimal solution $y^*(x)$ is linearly independent from the current collection of oracle optimal solutions $(y_k)_{k\in K}$. Indeed, this is explicitly stated in Algorithm 3 at line **??** and in Algorithm 5 at line **??**. In Algorithm 4, when a new index is added to K at line 9, we know that $v := S_B^\top W_B^{-1} T n_i \notin \text{span}\{(y_k)_{k\in K}\}$. But by hypothesis, the oracle has been called at the anchor point x° so $y^\circ = S_B^\top W_B^{-1} T(h - Tx^\circ) \in \text{span}\{(yk)_{k\in K}\}$. Therefore $y_{\text{new}} = y^\circ - v \notin \text{span}\{(y_k)_{k\in K}\}$.

This necessarily implies that the family of vectors $(y_k)_{k\in K}$ is linearly independent, hence the result. □

Proof of Proposition 8

Proof. We start the proof by stating a little technical lemma, that is a standard result in linear algebra.

Lemma 4. *[Dimension of the pre-image of a subspace]*
Let $W : \mathbb{R}^m \to \mathbb{R}^p$ be a linear map, and let $V \subseteq \mathbb{R}^p$ be a subspace. Then:

$$\dim\left(W^{-1}(V)\right) = m - \text{rank}(W) + \dim(V \cap \text{Im}(W)).$$

Proof. Recall first that the pre-image of a subspace through a linear map is always a subspace. Define the linear map

$$\begin{array}{rccc} \varphi : & W^{-1}(V) & \longrightarrow & V \\ & y & \longmapsto & Wy \end{array}$$

By construction: φ is linear and surjective onto $V \cap \text{Im}(W)$. The kernel of φ is equal to $\ker(W)$ because $\ker(W) \subseteq W^{-1}(V)$.

We obtain that $\dim\left(W^{-1}(V)\right) = \dim \ker(W) + \dim(V \cap \text{Im}(W))$ by applying the Rank-Nullity Theorem to φ. Hence the result after applying the Rank-Nullity Theorem to W. □

Now notice that the second-stage constraints impose that for every $k \in K$, $Wy_k = h - Tx_k \in V$. Hence, $y_k \in W^{-1}(V)$. Therefore, using the lemma above gives us :

$$\text{rank}(Y_K) := \dim(\underbrace{\text{span}\{(\text{y}_\text{k})_{\text{k}\in\text{K}}\}}_{\subseteq W^{-1}(V)}) \leq m - \text{rank}(W) + \dim(V \cap \text{Im}(W))$$

Hence the result. □